

Rapport de projet PGL GI1-D : Simulation d'agents et graphes

1. Introduction et présentation de l'équipe

Dans le cadre du module PGL (Projet Génie Logiciel) en ING1, notre équipe a été chargée de développer une application graphique Java permettant de visualiser, modifier et simuler le déplacement d'agents autonomes au sein d'un graphe.

Ce projet s'inscrit dans la continuité de notre travail en phase de Design, où nous avons imaginé une startup proposant des solutions de gestion de robots d'entrepôt logistique (optimisation des flux, algorithmes avancés, et respect du bien-être des employés). Dans notre application Java, le graphe représente les allées et postes de l'entrepôt, et les agents incarnent les robots de manutention.

L'équipe (Groupe D) est composée de :

- Ilias Mourtada - UI Interagir avec les éléments
- Tsiky Rivoherison - Propreté du code / Bonnes pratiques
- Adem Ben-Halima - Algorithme / Optimisations et Fonctionnalités / UI
- Corentin Pellerin - Polyvalent / Interagir avec le graph
- Antoine Arnoux - Algorithme / UI / Correcteur / Rapporteur

Notre tuteur pour ce projet était Sirine Hawari, avec qui nous avons organisé des points de suivi réguliers tout au long des 4 semaines de développement.

2. Gestion de projet et flux de travail (Workflow)

Pour assurer la livraison d'un Produit Minimum Viable (MVP) dès les premières semaines et itérer efficacement, nous avons adopté une méthodologie agile rythmée par des sprints hebdomadaires :

- **Suivi des tâches** : Nous avons utilisé **Jira** avec un tableau Kanban. Cela nous a permis de découper le projet en sous-tâches (création des classes **Node**, **Edge**, rendu graphique, algorithme de Dijkstra) et d'assigner les responsabilités.
- **Versionning** : Le code source a été centralisé sur un dépôt [GitHub](#). Nous avons mis en place une politique de branches pour développer les fonctionnalités en parallèle avant de les fusionner sur la branche principale, règles simples on fait ses taches sur sa branche et on ne push sur le main que du code fonctionnel.
- **Historique des Sprints** :
 - **Semaine 1 (19/05 - 26/05)** : Définition de l'architecture multicouche et implémentation du MVP (création dynamique du graphe, boucle temporelle **SimulationEngine**, rendu basique et algorithme de Dijkstra).

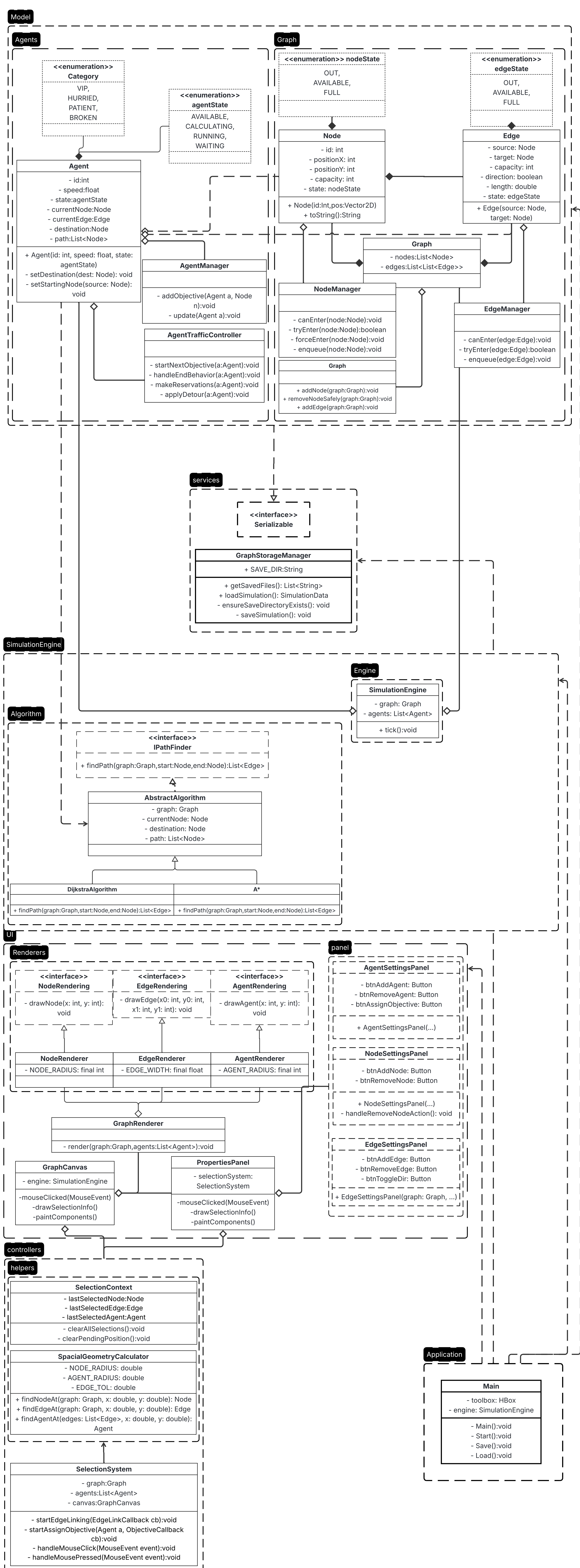
- **Semaine 2 (26/05 - 02/06)** : Correction de l'architecture (suppression des héritages abusifs), implémentation du système de files d'attente (queues) sur les arêtes et refonte des indicateurs visuels de l'UI.
- **Semaine 3 (02/06 - 05/06)** : Ajout des comportements avancés des agents (VIP, Pressé, Patient, Défaillant), amélioration de l'esthétique (dégradés de couleurs) et séparation stricte du moteur de simulation et de l'affichage.
- **Semaine 4 (05/06 - 10/06)** : Finalisation. Ajout du système d'import/export (sauvegarde binaire), calcul des statistiques (KPI), traduction du code en anglais et gestion robuste des exceptions (téléportation d'agents lors de la destruction d'une arête).

3. Architecture et conception technique

3.1 Architecture globale

Dès la conception, notre priorité était de garantir une séparation nette des responsabilités. Notre projet est structuré en plusieurs couches distinctes :

1. **Model (Modèle de graphe)** : Contient les données pures (**Graph**, **Node**, **Edge**, **Agent**). Ces classes ne connaissent rien de l'interface graphique.
2. **Simulation Engine (Moteur)** : Gère la boucle temporelle (système de *tick*). Il orchestre les déplacements, vérifie les capacités des arêtes (système de queue) et applique le Pathfinding (algorithme de Dijkstra) pour guider les agents vers leur destination.
3. **View & Controllers (UI)** : S'occupe du rendu graphique (**GraphRenderer**, affichage des nœuds et arêtes avec des couleurs dynamiques) et des interactions de l'utilisateur (sélection, ajout/suppression d'éléments).
4. **Services** : Contient le système d'Import/Export permettant de sauvegarder l'état exact de la simulation dans un fichier binaire pour le restaurer ultérieurement.



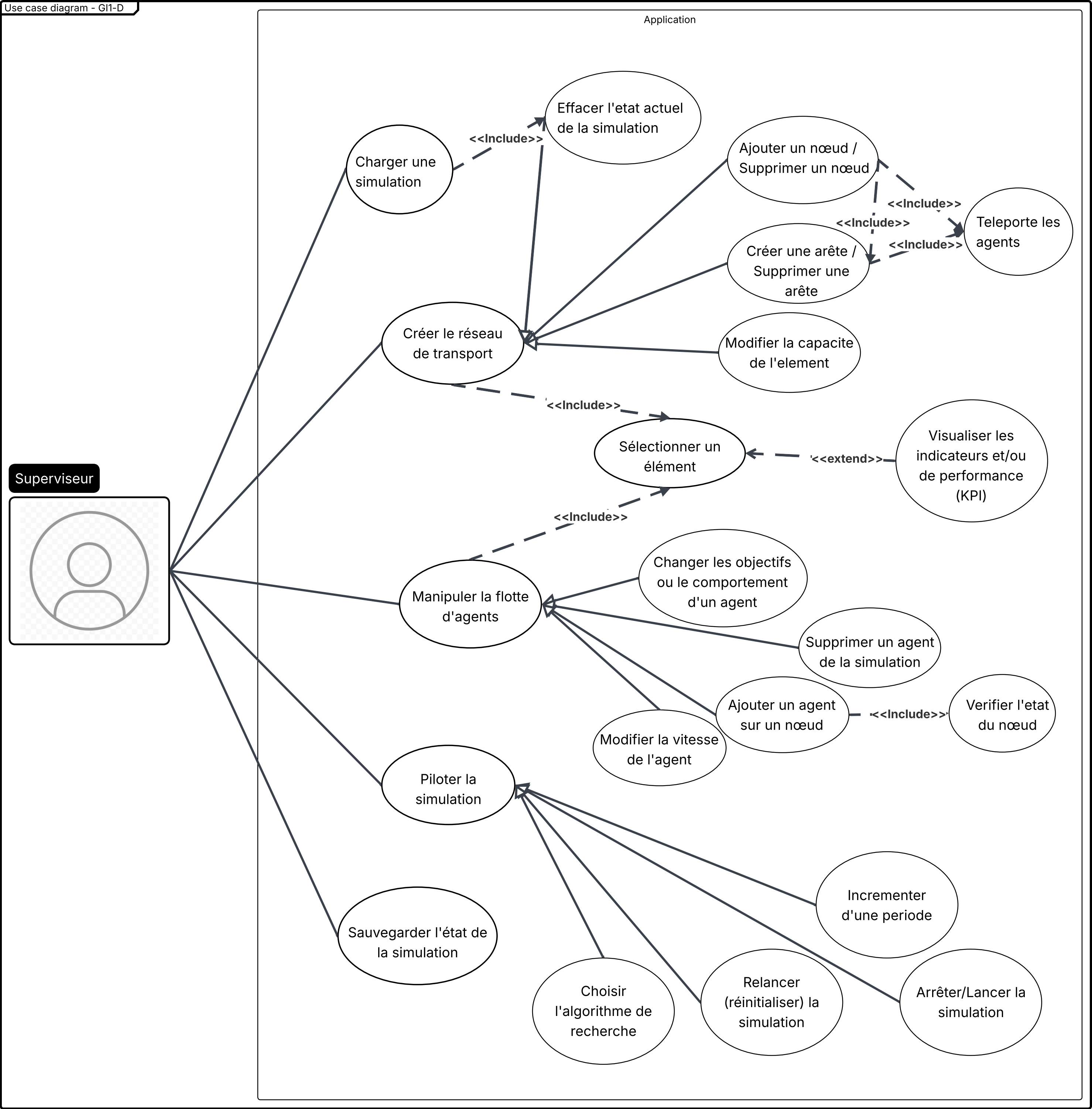
3.2 Description/Analyse des classes principales

- **Graph, Node, Edge** : Ces classes représentent la topologie d'un entrepôt. Node décrit des coordonnées, un état et une capacité. Edge décrit deux nœuds source et but ainsi qu'un état et une capacité. Graph structure tous ces éléments de manière logique avec une liste de nœuds, auxquels sont associées une liste d'arêtes adjacentes.
- **Agent** : L'agent est rigoureusement ancré dans le graph. Il décrit un nœud et/ou une arête, se caractérise par un état, un comportement et une vitesse.
- **SimulationEngine** : Le SimulationEngine s'occupe de faire la transition entre deux états du graph, à chaque itération il assigne les nouvelles positions et les nouveaux états.
- **Mécanisme de persistance et sauvegarde automatique (Autosave)** : L'application intègre un protocole de sauvegarde automatique à la fermeture. L'application suspend momentanément l'arrêt du processus pour capturer l'état structural du graphe et la liste active des agents. Ces données sont immédiatement encapsulées et sérialisées de manière transparente dans le fichier dévrouillé `autosave.sim`. Au redémarrage ultérieur de l'application, un mécanisme d'auto-load vérifie la présence de ce fichier pour restaurer fidèlement l'environnement de travail là où l'opérateur l'avait laissé, minimisant ainsi tout risque de perte de données.

3.3 Analyse fonctionnelle et Cas d'utilisation

Cas d'utilisation principaux implémentés :

- Lorsque l'utilisateur lance l'application il pourrait vouloir récupérer son travail, il dispose d'un système de sauvegarde qui lui permet de charger ou de sauvegarder un travail.
- L'utilisateur peut ajouter ou supprimer dynamiquement des nœuds et des arêtes, il peut également en modifier la capacité de, l'état ou le sens pour une arête, il peut également générer aléatoirement en masse des éléments ou supprimer la totalité des éléments.
- L'utilisateur peut ajouter ou supprimer des agents, il peut également leur assigner des objectifs, choisir leurs comportements et en modifier la vitesse.
- Possibilité de mettre en pause, d'arrêter la pause, pendant la pause l'utilisateur peut incrémenter la simulation d'une période, relancer.
- L'utilisateur peut à tout moment suivre des statistiques de sélection (KPI) des éléments du graphe en cliquant sur l'élément, il peut aussi choisir l'algorithme de recherche.



Relations avancées :

- Lors de la suppression d'un élément les agents qui le nécessitent sont automatiquement téléportés ailleurs sur le graph, il recalcule un nouveau chemin.
- Lors du chargement d'une sauvegarde l'état actuel de la simulation est automatiquement effacé.
- Lors de la suppression d'un nœud les arêtes voisines sont automatiquement effacées.
- Pour pouvoir modifier des éléments il faut d'abord les sélectionner, le menu d'ajout devient alors un menu de modification.

4. Fonctionnement de la simulation

4.1 Algorithme de Pathfinding

- L'algorithme de pathfinding est à la base de ce projet, c'est pourquoi nous l'avons implémenté dès la première semaine. Pour le MVP nous nous sommes contentés d'un dijkstra simple qui comprend l'initialisation, la phase de construction, la phase de validation et la phase de construction du chemin.
- Cela dit, cet algorithme présente de nombreux défauts dans un système multi agent: il ne prend pas en compte la congestion des nœuds et arêtes. Il optimise la distance pure plutôt que le temps de trajet. Il envoie tous les agents dans la même direction provoquant ainsi des goulots d'étranglements, ralentissements et blocages.
- Afin que tous les agents ne se précipitent pas dans la même direction, nous avons implémenté un système de réservation qui augmente le poids des arrêts selon la demande. De plus, le taux de remplacement d'un nœud ou arête fait également augmenter le poids du chemin. Avec ces modifications il est possible d'estimer le temps de trajet de chaque agent et de faire un "Dijkstra temporel".
- Finalement, le système de pathfinding permet de choisir entre A* et Dijkstra tous deux améliorés. Cela permet de comparer les performances KPI, de plus la structure interface pathfinding, classe abstraite algo permet d'y insérer n'importe quel autre algorithme de recherche pour faire des comparaisons.

4.2 Dynamique de déplacement et comportement des agents

- Les agents sont rigoureusement ancrés dans le graph. À tout moment, ils sont positionnés sur une arête et/ou un nœud et ils disposent également d'une distance sur l'arête. La position des agents sur le plan n'est donc qu'une projection de la position des agents dans le graph.
- De plus les agents se caractérisent par différentes priorités or plus la priorité est haute plus le chemin est assigné à tout, ce qui mécaniquement fait que plus la priorité est haute plus le chemin est court.
- En début de simulation un agent est AVAILABLE, à l'assignation d'un objectif il calcule son chemin et se lance, il est RUNNING, lorsque l'agent n'a plus d'objectifs il termine et est OUT.

- En cas de blocage : Les agents vont attendre dans une file d'attente, perdre patience et essayer de débloquent la situation en se décalant.
- En cas d'arête supprimée l'agent va recalculer son chemin, les agents présents sur l'arête supprimée sont téléportés dans leurs nœuds source et calculent leurs chemin.
- Les agents disposons de différents comportements : VIP(les autres agents s'écartent pour le laisser passer, il est plus rapide), HURRIED(il est plus rapide, moins patient), PATIENT(c'est le comportement par défaut), BROKEN(il a un comportement défaillant qui crée des blocages).

5. Problèmes rencontrés, solutions et design inclusif

Durant le développement, nous avons été confrontés à plusieurs défis architecturaux et algorithmiques qui nous ont forcés à refactoriser notre code :

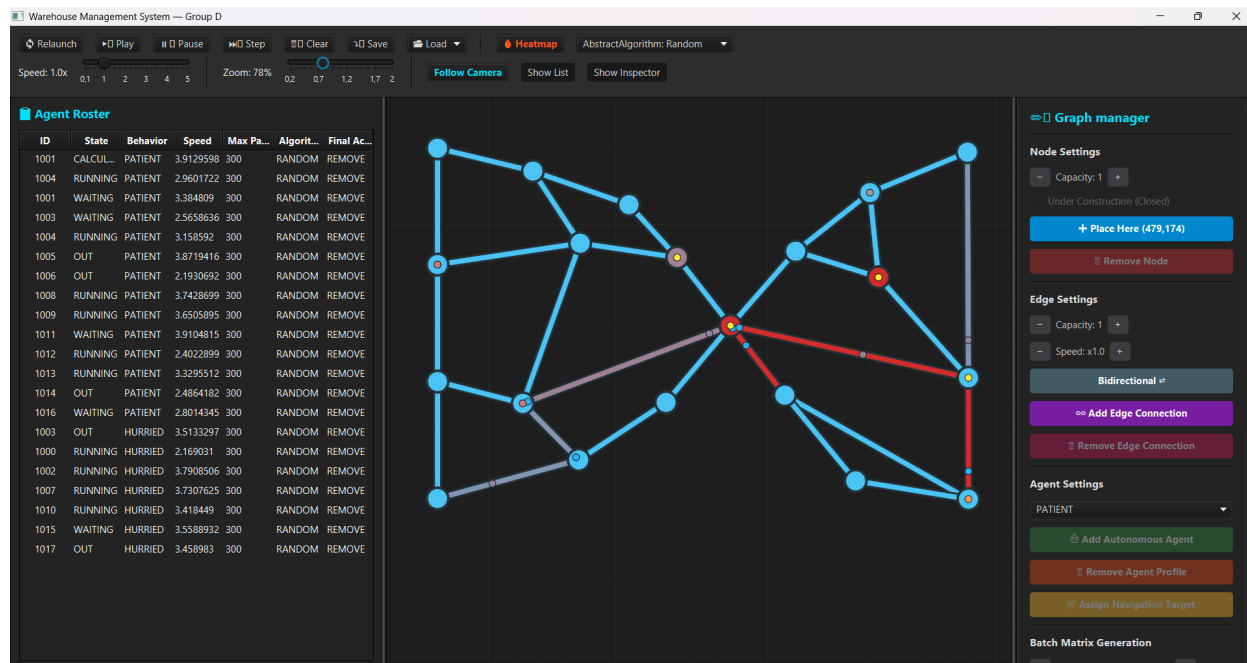
Problème / Bug identifié	Solution technique apportée
Couplage fort entre Vue et Modèle : Initialement, le <code>SimulationEngine</code> appelait directement la méthode <code>draw()</code> de l'interface.	Découplage : Le moteur de simulation a été rendu indépendant. C'est désormais le contrôleur de l'UI qui observe la simulation et déclenche le rafraîchissement graphique.
Mauvaise utilisation de l'héritage : La classe <code>Agent</code> héritait de <code>Dijkstra</code> (<code>extends Dijkstra</code>), ce qui n'avait pas de sens conceptuel.	Composition : L'algorithme a été extrait dans une interface <code>PathFinder</code> intégrée directement au sein du moteur de simulation, permettant aux agents de simplement interroger le moteur pour leur route.
Blocage entre agents : Si un agent s'engage sur une route déjà saturée, il aggrave irrémédiablement le blocage.	Anticipation & Smart Waiting : Avant de s'engager, la méthode <code>isPathClearAhead()</code> vérifie l'état d'occupation des 2 prochains segments du chemin de l'agent. Si un goulot d'étranglement est détecté, l'agent se met proactivement en pause avec le log <i>"Anticipating a traffic jam, doing SMART WAITING"</i>. Réservation spatiale : Dès qu'un agent calcule un itinéraire, il signale sa future présence via <code>makeReservations()</code> en incrémentant la variable

	<code>expectedOccupants</code> sur les nœuds et arêtes de son trajet.
Blocage statique (Deadlock) sur une intersection: Deux agents ou plus se bloquent mutuellement à une intersection (Nœud) de manière indéfinie.	Échappatoire : Lorsque la patience tombe à zéro, la méthode <code>applyDetour()</code> est appelée. L'agent cherche instantanément un nœud adjacent valide pour contourner le bouchon, ou force un recalcul total de son chemin (A*/Dijkstra) si aucune issue adjacente n'est disponible.
Face-à-face et blocage au milieu d'une arête (route): Un agent est engagé sur une arête mais ne peut plus avancer car la destination est bloquée.	Marche arrière d'urgence / Retreating : Si la patience de l'agent expire alors qu'il est déjà sur une arête (<code>getCurrentEdge() != null</code>), le système active le mode de repli avec <code>setRetreating(true)</code> La physique de la fonction <code>processEdgeMovement()</code> s'inverse alors : la vitesse de déplacement (<code>distMoved</code>) est soustraite, forçant le véhicule à faire physiquement marche arrière jusqu'à son nœud de départ pour libérer la route.
L'algorithme crée lui-même des goulots d'étranglement: L'algorithme de base de Dijkstra favorise tous les agents sur la même route la plus courte.	Pathfinding Dynamique Pondéré : Les algorithmes <code>AStar</code> et <code>Dijkstra</code> ne se basent pas uniquement sur la distance. Le coût d'un déplacement (<code>dynamicCost</code>) intègre une très lourde pénalité (<code>TRAFFIC_PENALTY = 5000.0</code>) qui est multipliée par le nombre d'occupants attendus (<code>expectedOccupants</code>) sur la route et sur le nœud visé. De cette manière, les chemins se répartissent organiquement pour éviter les zones à risque.

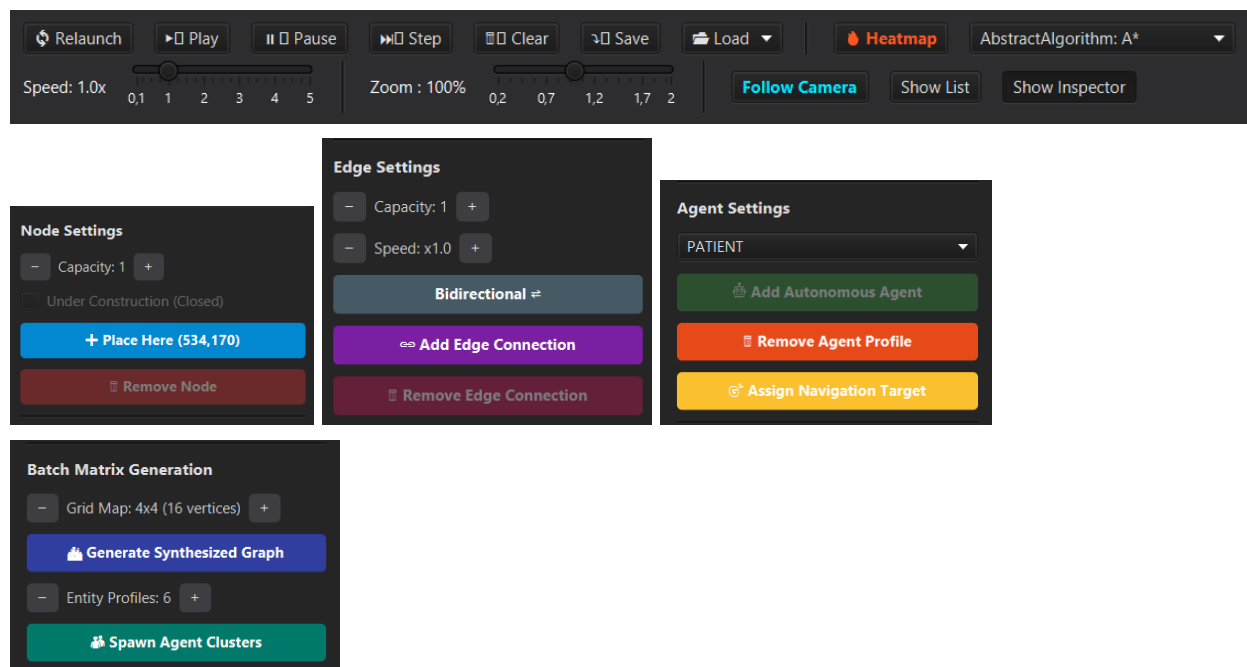
Le bug d'affichage en mode "Pause" :Lorsque la simulation était mise en pause, cliquer sur un élément pour le sélectionner ne déclenchait aucun halo visuel car la boucle de rafraîchissement graphique (liée au "tick" du moteur) était stoppée.	Mise en place d'un forçage du rendu (<code>redraw()</code>) déclenché spécifiquement par les clics de souris, de sorte que l'interface reste interactive même quand le temps de la simulation est figé.
L'agent bloqué sur une route fantôme : Lorsqu'une arête située <i>plus loin</i> sur le chemin d'un agent était supprimée par l'utilisateur, l'agent continuait sa route (état RUNNING) mais finissait par faire du sur-place dans le vide.	Implémentation d'un système de recalcul d'itinéraire. L'agent vérifie la validité de sa route en temps réel et relance Dijkstra/A* si son trajet a été détruit.
Crash lors de la suppression d'éléments : Si un utilisateur supprimait un nœud ou une arête pendant qu'un agent s'y trouvait, le programme plantait (NullPointerException).	Sécurité et Téléportation : Implémentation d'une suppression sécurisée avec nettoyage des arêtes liées. Si un élément est détruit sous un agent, ce dernier est géré par une fonction de "téléportation" sécurisée ou remplacée sur le nœud valide le plus proche.
Les graphes non connexes (villes coupées en deux) : Si le graphe était séparé en deux blocs sans route pour les relier, l'algorithme de Dijkstra cherchait indéfiniment des liens inexistantes pour atteindre sa cible, ce qui causait un crash.	Sécurisation de l'algorithme : si aucun chemin n'existe, le pathfinding s'arrête proprement et renvoie une liste vide. L'agent abandonne alors son objectif et en demande un nouveau.

Crash lorsque Dijkstra ne trouve pas de chemin : Si un graph avait deux composantes non connexes alors Dijkstra cherchait des liens qui n'existaient pas : (NullPointerException).	1er approche: lorsqu'un graph(orienté) est de la forme: <table border="1" data-bbox="933 304 1245 543"> <tr> <th>Noeud</th><th>Noeuds voisins</th></tr> <tr> <td>A</td><td>C, B</td></tr> <tr> <td>B</td><td></td></tr> <tr> <td>C</td><td>A, B</td></tr> </table> Si le chemin n'existe pas on renvoie un chemin vide. Amélioration : Nous avons pensé attribuer l'objectif a un agent présent dans l'autre composante du graph.	Noeud	Noeuds voisins	A	C, B	B		C	A, B
Noeud	Noeuds voisins								
A	C, B								
B									
C	A, B								
Bug lorsqu'on supprime une arête sur le chemin d'un agent : Lors de la suppression d'une arête du chemin d'un agent. L'agent continuer sa route*(etat: RUNNING) mais sans avancer.	Recalcul d'itinéraire : Lorsqu'un agent fait du sur place dans l'état RUNNING, il recalcule maintenant son chemin et s'il existe il l'emprunte.								
Lisibilité du code et normes : Mélange de français et d'anglais dans le terminal et les commentaires.	Standardisation : Lors de la semaine 4, une passe complète de nettoyage (Clean Code) a été effectuée pour tout traduire en anglais et commenter les classes complexes.								
Éléments invisibles : Afin de garantir une accessibilité universelle de l'interface, les palettes de couleurs ont été adaptées pour assurer une parfaite lisibilité, notamment pour les utilisateurs atteints de dyschromatopsie (daltonisme).	Refonte visuelle inclusive : Une mise à jour des couleurs a été faite sur les renderers afin que l'UI soit intelligible pour tous types de voyant.								

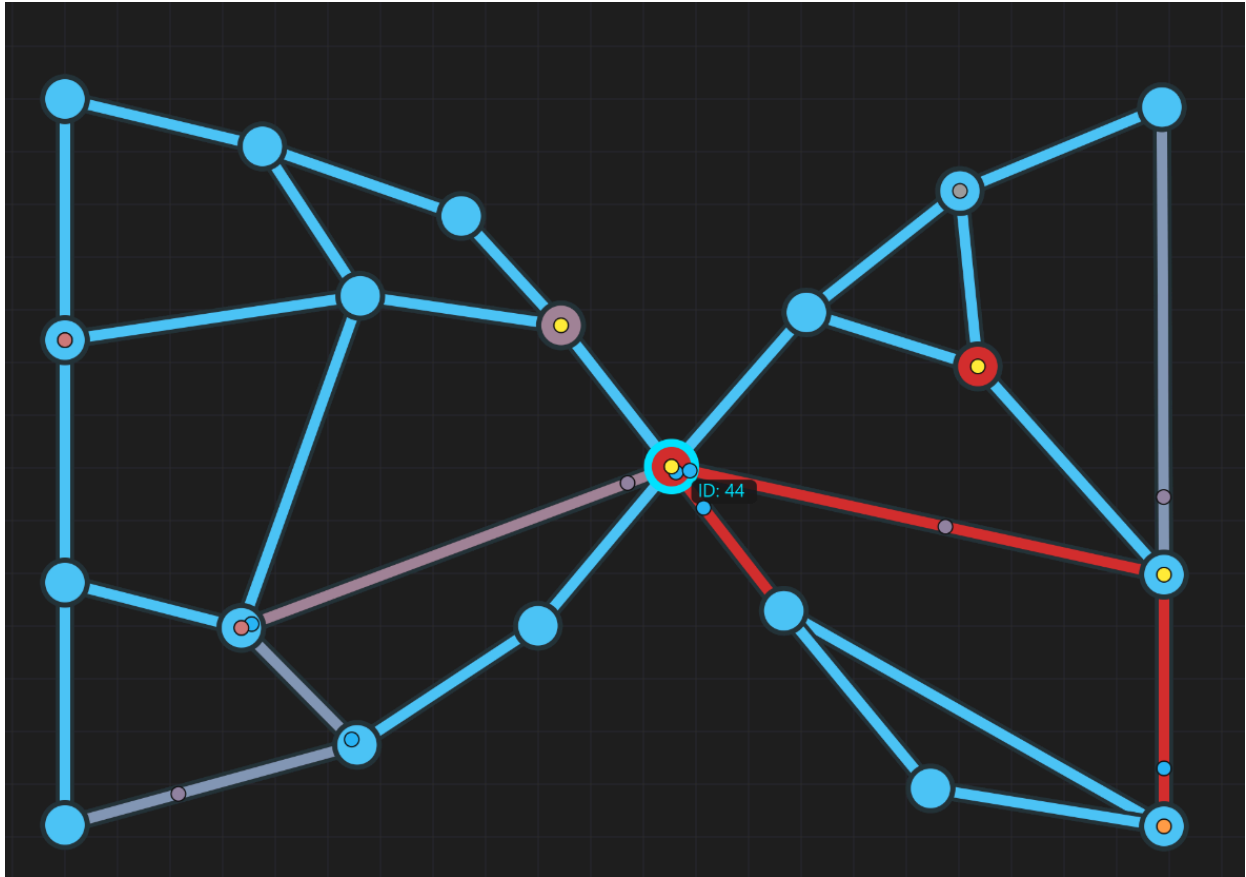
6. Résultats et Conclusion



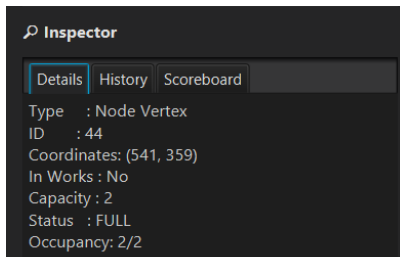
Lors du lancement de l'application, le dernier graph sauvegarde est automatique charger sinon c'est un graph de démonstration qui est initialisé. On peut egalement modifier le graph.



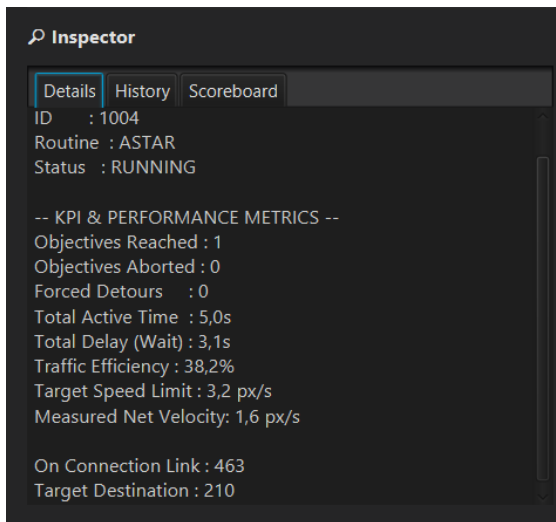
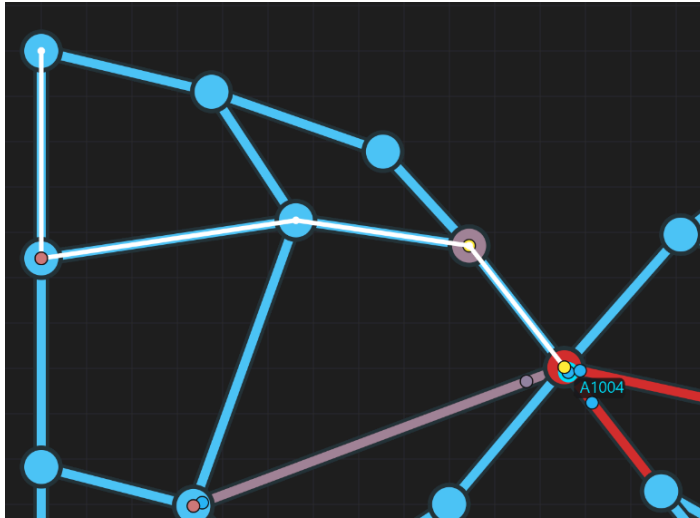
L'interface graphique permet de voir la congestion et les goulots d'étranglements du graph par la coloration des éléments avec un gradient de couleurs, les goulots d'étranglements deviennent très facile à repérer.



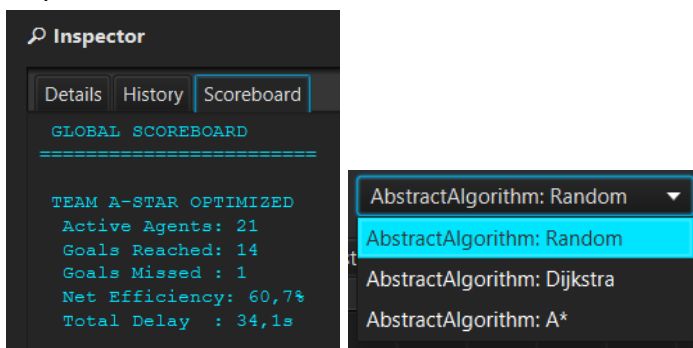
//Capture d'écran de l'application : On y voit un graph doté d'un unique noeud central de degre pondere 10 mais de capacite21. Les aretes les plus directes sont aussi moins rapide. Comment notre systeme va t-il reagir ?



Elle permet aussi d'afficher le chemin que parcourt un agent lorsqu'il est sélectionné.



La possibilité de changer dynamiquement l'algorithme de calcul de trajectoire (Pathfinding) via l'interface transforme le tableau de bord en un véritable outil d'analyse comparative. L'utilisateur peut observer l'impact direct du choix algorithmique et de la forme du graph sur les indicateurs de performance.



À l'issue de ces 4 semaines de développement intense, nous avons livré un produit qui respecte pleinement le cahier des charges initial et notre vision "Design".

L'application permet non seulement de modéliser un graphe fonctionnel, mais aussi d'y observer des flux complexes. Les différents comportements implantés (VIP/Pressé, Patient, Défaillant) génèrent des situations de congestion réalistes que l'utilisateur peut analyser grâce aux KPI et aux indicateurs visuels colorés (dégradés). L'ajout de l'import/export binaire rend l'outil concrètement utilisable pour concevoir des scénarios à l'avance.

Ce projet a été extrêmement formateur sur l'importance d'une architecture logicielle réfléchie (MVC), de la bonne gestion des versions avec Git, et de la communication continue au sein d'une équipe de développement.